

DSC 40B

Lecture 10 : Sorting

Sorting

Sorting

- Sorting is a **very** common operation.
- But **why** is it important?
- A e s t h e t i c reasons?
- Sorting makes some problems easier to solve.

Today (and next Lecture)

- How do we sort?
- How fast can we sort?
- How do we use sorted structure to write faster algorithms?
- **Also**: how to understand complex loops with **loop invariants**.

Selection Sort

Selection Sort

- Repeatedly remove **smallest** element.
- Put it at the **end** of new list.

Example

Example: arr = [7, 8, 5, 3, 1]

Example

Example: arr = [7, 8, 5, 3, 1]

- []

Example

Example: arr = [7, 8, 5, 3, 1]

- [1,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5, 7,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5, 7,]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5, 7, 8]

Example

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5, 7, 8]

Example. Issue?

Example: arr = [7, 8, 5, 3, 4]

- [1, 3, 5, 7, 8]

In-place Selection Sort

- We **don't need** a separate list.
 - We can swap elements until sorted.
- Store "new" list at the beginning of input list.
- Separate the old and new with a **barrier**.

Example: sorting in-place

Example: `arr = [7, 8, 5, 3, 1]`

Example: sorting in-place

Example: arr = [7, 8, 5, 3, 1]

[7, 8, 5, 3, 1]

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: `arr = [7, 8, 5, 3, 1]`

`[7, 8, 5, 3, 1]`

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: `arr = [7, 8, 5, 3, 1]`

`[7, 8, 5, 3, 1]` *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: `arr = [7, 8, 5, 3, 1]`

`[7, 8, 5, 3, 1]` *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: `arr = [7, 8, 5, 3, 1]`

`[1, 8, 5, 3, 7]` *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 8, 5, 3, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 8, 5, 3, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 8, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 8, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 8, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 8, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 8, 7] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 7, 8] *#swap, move the barrier*

Example: sorting in-place

Left: sorted.

Right: needs to be sorted

Example: arr = [7, 8, 5, 3, 1]

[1, 3, 5, 7, 8] *#swap, move the barrier*

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

```
def find_minimum(arr, start):  
    """Finds index of minimum. Assumes non-empty."""  
    n = len(arr)  
    min_value = arr[start]  
    min_ix = start  
    for i in range(start + 1, n):  
        if arr[i] < min_value:  
            min_value = arr[i]  
            min_ix = i  
    return min_ix
```

Loop Invariants

- How do we understand an iterative algorithm?
- A **loop invariant** is a statement that is **true** after every iteration.
 - And before the loop begins!

Loop Invariant(s)

- After the α th iteration of selection sort, each of the first α elements is $\leq$ each of the remaining elements.

Example: `arr = [7, 8, 5, 3, 1]`

Loop Invariant(s)

- After the α th iteration of selection sort, each of the first α elements is $\leq$ each of the remaining elements.
- $\alpha = 0$: arr = [7, 8, 5, 3, 1] #empty set. True
- $\alpha = 1$: arr = [1, 8, 5, 3, 7] # 1 is $\leq$
- $\alpha = 2$: arr = [1, 3, 5, 8, 7] # 1, 3 are $\leq$

Loop Invariant(s)

- After the α th iteration of selection sort, each of the first α elements is $\leq$ each of the remaining elements.
- $\alpha = 0$: arr = [7, 8, 5, 3, 1] #empty set. True
- $\alpha = 1$: arr = [1, 8, 5, 3, 7] # 1 is $\leq$
- $\alpha = 2$: arr = [1, 3, 5, 8, 7] # 1, 3 are $\leq$

Is it enough to claim that the algorithm works properly?

A: Yes

B: No

Loop Invariant(s)

- After the α th iteration of selection sort, each of the first α elements is $\leq$ each of the remaining elements.
- $\alpha = 0$: arr = [7, 8, 5, 3, 1] #empty set. True
- $\alpha = 1$: arr = [1, 8, 5, 3, 7] # 1 is $\leq$
- $\alpha = 2$: arr = [3, 1, 5, 8, 7] # 1, 3 are $\leq$

Loop Invariant(s)

- After the α th iteration, the first α elements are **sorted**.

Loop Invariants

- Plug the total number of iterations into the loop invariant to learn about the results.
 - `selection_sort` makes $n - 1$ iterations:
 - After the $(n - 1)$ th iteration, the first $(n - 1)$ elements are sorted.
 - After the $(n - 1)$ th iteration, each of the first $(n - 1)$ elements is $\leq$ each of the remaining elements.

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[barrier_ix:]  
        min_value = arr[barrier_ix]  
        min_ix = barrier_ix  
        for i in range(barrier_ix + 1, n):  
            if arr[i] < min_value:  
                min_value = arr[i]  
                min_ix = i  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[barrier_ix:]  
        min_value = arr[barrier_ix]  
        min_ix = barrier_ix  
        for i in range(barrier_ix + 1, n):  
            if arr[i] < min_value:  
                min_value = arr[i]  
                min_ix = i  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[barrier_ix:]  
        min_value = arr[barrier_ix]  
        min_ix = barrier_ix  
        for i in range(barrier_ix + 1, n):  
            if arr[i] < min_value:  
                min_value = arr[i]  
                min_ix = i  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

(n-1) + (n-2)

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[barrier_ix:]  
        min_value = arr[barrier_ix]  
        min_ix = barrier_ix  
        for i in range(barrier_ix + 1, n):  
            if arr[i] < min_value:  
                min_value = arr[i]  
                min_ix = i  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

$(n-1) + (n-2) + (n-3)$


```
def selection_sort(arr):  
    """In-place selection sort."""
```

```
    n = len(arr)
```

```
    if n <= 1:
```

```
        return
```

```
    for barrier_ix in range(n-1):
```

```
        # find index of min in arr[barrier_ix:]
```

```
        min_value = arr[barrier_ix]
```

```
        min_ix = barrier_ix
```

```
        for i in range(barrier_ix + 1, n):
```

```
            if arr[i] < min_value:
```

```
                min_value = arr[i]
```

```
                min_ix = i
```

```
        #swap
```

```
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

$(n-1) + (n-2) + (n-3) + \dots + 1$

```
def selection_sort(arr):  
    """In-place selection sort."""
```

```
    n = len(arr)
```

```
    if n <= 1:
```

```
        return
```

```
    for barrier_ix in range(n-1):
```

```
        # find index of min in arr[barrier_ix:]
```

```
        min_value = arr[barrier_ix]
```

```
        min_ix = barrier_ix
```

```
        for i in range(barrier_ix + 1, n):
```

```
            if arr[i] < min_value:
```

```
                min_value = arr[i]
```

```
                min_ix = i
```

```
        #swap
```

```
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

$$(n-1) + (n-2) + (n-3) + \dots + 1 = \Theta(n^2)$$

Time Complexity

- Selection sort takes $\Theta(n^2)$ time.

Time Complexity for Selection Sort

- How about sorted array input?

A: $\Theta(1)$ time.

B: $\Theta(n)$ time.

C: $\Theta(n^2)$ time.

D: Something else

*Modify selection_sort so that it computes a **median** of the input array. What is the time complexity?*

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```



Thank you!

Do you have any questions?

CampusWire!